

AWS EC2 Deployment

Complete Guide with Issues & Solutions

Project	sameer-ai-platform (Nuh Bot)
Owner	Sameer Shaik
Company	N.S (Nadeem Shaik)
AI Engine	Groq API — llama-3.3-70b-versatile
Region	Asia Pacific — Sydney (ap-southeast-2)
Instance	m7i-flex.large (2 vCPU, 8GB RAM)
OS	Ubuntu 26.04 LTS
Services	Spring Boot + Node.js + Python + Open WebUI
Date	June 2026

This document is a complete step-by-step deployment guide for the Nuh AI WhatsApp Bot Platform on AWS EC2. It covers every step from EC2 instance creation to running all Docker containers, including all technical issues encountered during deployment and their exact solutions with commands.

Table of Contents

- Step 1** — EC2 Instance Setup: *Configuration choices and security setup*
- Step 2** — SSH Connection: *Connecting to the server*
- Step 3** — Server Update & Reboot: *System update after launch*
- Step 4** — Docker Installation: *Installing Docker with issue fix*
- Step 5** — Java & Maven Installation: *Build tools setup*
- Step 6** — Ollama Installation: *AI model runtime setup*
- Step 7** — Nuh Model Creation: *Custom AI model setup*
- Step 8** — Project Transfer: *Uploading project to EC2*
- Step 9** — Build & Deploy: *Building and running all containers*
- Step 10** — Issue: Ollama Not Reachable: *Docker network fix*
- Step 11** — Issue: Slow CPU Response: *Switching to Groq API*
- Step 12** — Issue: Wrong Groq URL: *API endpoint fix*
- Step 13** — Issue: Deprecated Model: *Model name update*
- Step 14** — Final Test & Verification: *Complete system test*
- Step 15** — Daily Management Commands: *Start, stop, monitor*

Step 1 — EC2 Instance Setup (AWS Console)

1.1 Instance Configuration

Setting	Value Chosen	Reason
Name	Learn_from_Nuh	Project identifier
AMI	Ubuntu 26.04 LTS	Stable, free tier eligible
Instance Type	m7i-flex.large	2 vCPU, 8GB RAM — needed for Docker + Ollama
Key Pair	sameer-nuh-key (RSA .pem)	Required for SSH access
Storage	30 GiB gp3 SSD	Fast SSD, enough for all Docker images + models
Purchasing	On-Demand	Pay per hour, stop anytime to save cost

1.2 Security Group Rules

Created security group with 4 inbound rules — all limited to My IP for security:

Rule	Port	Source	Description
SSH	22	My IP only	Admin access — SSH for Sameer admin access
Custom TCP	8080	My IP only	Spring AI Gateway
Custom TCP	8000	My IP only	Python block service
Custom TCP	3000	My IP only	Open WebUI dashboard

■ ■ Warning: NEVER set SSH source to 0.0.0.0/0 (Anywhere). Always use My IP only for SSH!

1.3 Key Pair

Created key pair: **sameer-nuh-key** — Downloaded sameer-nuh-key.pem to Downloads folder.

■ ■ Warning: Save .pem file safely — you can NEVER download it again from AWS!

Move it immediately after download:

```
cp ~/Downloads/sameer-nuh-key.pem ~/Documents/sameer-nuh-key.pem chmod 400  
~/Documents/sameer-nuh-key.pem
```

1.4 Advanced Details

Setting	Value	Why
Termination Protection	Enable	Prevents accidental permanent deletion
Stop Protection	Enable	Prevents accidental stop
Shutdown Behavior	Stop	If OS shuts down, instance stops not terminates
File Systems	None	Single server — no shared storage needed
Purchasing Option	On-Demand	Flexibility to stop anytime

Step 2 — SSH Into EC2 Instance

After instance launches, get the Public IP from EC2 Console → Instances.

EC2 Public IP: **3.104.66.117**

Commands (run on your laptop terminal):

```
# Fix key permissions first chmod 400 ~/Downloads/sameer-nuh-key.pem # Connect to EC2 ssh -i ~/Downloads/sameer-nuh-key.pem ubuntu@3.104.66.117
```

When prompted:

```
Are you sure you want to continue connecting (yes/no)? yes
```

```
■ Connected! Prompt changes to: ubuntu@ip-172-31-39-34:~$
```

```
■ Note: ubuntu@ip-172-31-39-34 = EC2 server | sameer-shaik@IdeaPad = your laptop. Never confuse the two!
```

Step 3 — Update Server & Reboot

Run on EC2:

```
sudo apt update && sudo apt upgrade -y
```

After update completes, a message appears: *Pending kernel upgrade! Restarting the system to load the new kernel.*

Reboot to apply kernel:

```
sudo reboot
```

SSH drops — wait 30 seconds then reconnect:

```
ssh -i ~/Downloads/sameer-nuh-key.pem ubuntu@3.104.66.117
```

```
■ Kernel updated from 7.0.0-1004-aws to 7.0.0-1006-aws
```

Step 4 — Docker Installation (with Issue Fix)

■ ISSUE 1: docker-compose-plugin not found in Ubuntu 26.04

```
■ ERROR: sudo apt install docker.io docker-compose-plugin -y Error: Unable to locate package docker-compose-plugin
```

Ubuntu 26.04 does not have docker-compose-plugin in its default apt repository. Must add Docker's official repository manually.

■ SOLUTION: Add Docker's Official Repository

```
# Step 1 — Install required packages sudo apt install ca-certificates curl -y
```

```
# Step 2 — Add Docker GPG key sudo install -m 0755 -d /etc/apt/keyrings sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
# Step 3 — Add Docker repository echo \ "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \ https://download.docker.com/linux/ubuntu noble stable" | \ sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
# Step 4 — Install Docker sudo apt update sudo apt install docker-ce docker-ce-cli containerd.io docker-compose-plugin -y
```

```
# Step 5 — Add user to docker group sudo usermod -aG docker ubuntu newgrp docker
```

```
# Step 6 — Verify docker --version docker compose version
```


Step 5 — Java & Maven Installation

```
sudo apt install openjdk-21-jdk maven -y
```

```
# Verify java -version mvn -version
```

■ OpenJDK 21.0.11 | Apache Maven 3.9.12

Step 6 — Ollama Installation & Configuration

Install Ollama:

```
curl -fsSL https://ollama.com/install.sh | sh
```

■ Note: WARNING: No NVIDIA/AMD GPU detected. Ollama will run in CPU-only mode. (This caused issues later — see Step 11)

```
sudo systemctl enable ollama sudo systemctl start ollama sudo systemctl status ollama
```

■ Ollama active (running) on port 11434

Configure Ollama to listen on all interfaces (required for Docker):

```
sudo mkdir -p /etc/systemd/system/ollama.service.d sudo tee  
/etc/systemd/system/ollama.service.d/override.conf <<< 'EOF' [Service]  
Environment="OLLAMA_HOST=0.0.0.0" EOF
```

```
sudo systemctl daemon-reload sudo systemctl restart ollama ss -tlnp | grep 11434
```

■ LISTEN *:11434 — Ollama now reachable from Docker containers

Step 7 — Create Nuh Custom Model

```
# Pull base model ollama pull qwen2.5:1.5b
```

```
# Create Modelfile cat > /tmp/Modelfile <<< 'EOF' FROM qwen2.5:1.5b SYSTEM "" You are  
Nuh, an AI assistant. Version: Nuh 1.0 Created by: Sameer Shaik Company: N.S Keep replies  
short. Plain text only. "" PARAMETER temperature 0.7 PARAMETER top_p 0.9 EOF
```

```
# Create Nuh model ollama create Nuh -f /tmp/Modelfile # Verify ollama list
```

■ Nuh:latest 986MB created successfully

Step 8 — Transfer Project to EC2

■ ISSUE 2: SCP failing with thousands of small files (WhatsApp auth folder)

```
■ ERROR: scp -r ~/Desktop/sameer-ai-platform ubuntu@3.104.66.117:~/ # Connection drops after thousands of small files # Error: client_loop: send disconnect: Broken pipe
```

The WhatsApp auth folder contains thousands of tiny JSON files (pre-keys, session files, lid-mappings). Transferring file-by-file causes SSH connection timeout.

■ SOLUTION: Zip first, transfer one file

```
# On LAPTOP terminal – zip project (exclude node_modules) cd ~/Desktop zip -r sameer-ai-platform.zip sameer-ai-platform \ --exclude '*/node_modules/*' \ --exclude '*/.venv/*'
```

```
# Transfer single zip file scp -i ~/Downloads/sameer-nuh-key.pem \ ~/Desktop/sameer-ai-platform.zip \ ubuntu@3.104.66.117:~/
```

```
# On EC2 – unzip (use -o flag to avoid per-file confirmation) sudo apt install unzip -y unzip -o ~/sameer-ai-platform.zip -d ~/ ls ~/sameer-ai-platform/
```

■ Project transferred: docker-compose.yml, services/ (ai-gateway, whatsapp-bot, python-service)

■ Note: The -o flag in unzip means 'overwrite all without asking' — avoids typing Y/N for every file

Step 9 — Build Spring Jar & Deploy

Build Spring jar (must be done on EC2 before Docker build):

```
cd ~/sameer-ai-platform/services/ai-gateway mvn clean package -DskipTests -q echo "Build: $?"  
ls target/*.jar
```

■ Build: 0 | target/ai-gateway-0.0.1-SNAPSHOT.jar

Start all Docker containers:

```
cd ~/sameer-ai-platform docker compose up -d docker compose ps
```

■ All 4 containers started: ai-gateway, whatsapp-bot, python-service, open-webui

Step 10 — Issue: Ollama Not Reachable from Docker

■ ISSUE 3: Spring cannot connect to Ollama inside Docker

```
■ ERROR: curl http://localhost:8080/api/ai/chat?message=hello {"status":500,"error":"Internal Server Error"}
```

Docker containers cannot use 'localhost' or 'host.docker.internal' to reach Ollama on EC2. The docker-compose.yml had:

```
SPRING_AI_OLLAMA_BASE_URL: http://host.docker.internal:11434
```

On EC2 Linux, host.docker.internal does not resolve correctly. Need the actual Docker bridge gateway IP.

■ SOLUTION: Find actual Docker gateway IP

```
# Find Docker network gateway docker network inspect sameer-ai-platform_default | grep Gateway
```

Output showed: "Gateway": "172.18.0.1"

```
# Verify Ollama is reachable on that IP curl http://172.18.0.1:11434/api/tags
```

```
■ Ollama responded with model list — reachable on 172.18.0.1
```

```
# Update docker-compose.yml with correct IP sed -i  
's|http://host.docker.internal:11434|http://172.18.0.1:11434|g' \  
~/sameer-ai-platform/docker-compose.yml
```

```
# Also update application.properties in Spring sed -i  
's|http://localhost:11434|http://172.18.0.1:11434|g' \  
~/sameer-ai-platform/services/ai-gateway/src/main/resources/application.properties # Verify  
cat ~/sameer-ai-platform/services/ai-gateway/src/main/resources/application.properties
```

```
# Rebuild jar and restart cd ~/sameer-ai-platform/services/ai-gateway mvn clean package  
-DskipTests -q cd ~/sameer-ai-platform docker compose down docker compose up -d --build
```

■ Note: Why 172.18.0.1 not 172.17.0.1? Docker creates a new bridge network for each compose project. The project network uses 172.18.x.x range, not the default docker0 bridge.

Step 11 — Issue: Extremely Slow AI Response (CPU-Only Mode)

■ ISSUE 4: Ollama running on CPU — too slow for WhatsApp bot

```
■ ERROR: curl http://localhost:8080/api/ai/chat?message=hello # Takes 60-120 seconds to respond # WhatsApp bot showing: AI server offline
```

The m7i-flex.large has no GPU. Ollama running qwen2.5:1.5b in CPU-only mode takes 60-120 seconds per response — completely unusable for a real-time WhatsApp bot.

Option	Speed	Cost	Decision
Keep CPU Ollama	Very slow (60-120s)	Free	Rejected — unusable
GPU EC2 (g4dn.xlarge)	Fast	\$0.50/hr	Rejected — too expensive
Groq API (free)	Very fast (1-2s)	Free tier	CHOSEN
OpenAI API	Fast	\$0.01/msg	Alternative option

■ SOLUTION: Switch to Groq API (Free & Fast)

Groq provides free API access to powerful models. Got API key from console.groq.com

Step 1 — Update application.properties to use Groq:

```
cat > ~/sameer-ai-platform/services/ai-gateway/src/main/resources/application.properties
<< 'EOF' spring.application.name=ai-gateway server.port=8080 # Groq API config
spring.ai.openai.api-key=YOUR_GROQ_API_KEY
spring.ai.openai.base-url=https://api.groq.com/openai/v1
spring.ai.openai.chat.options.model=llama-3.3-70b-versatile
spring.ai.openai.chat.options.temperature=0.7 EOF
```

Step 2 — Update pom.xml to use OpenAI starter instead of Ollama:

```
sed -i 's|spring-ai-starter-model-ollama|spring-ai-starter-model-openai|g' \
~/sameer-ai-platform/services/ai-gateway/pom.xml
```

■ Note: Groq API is compatible with OpenAI's API format, so we use Spring AI's OpenAI starter to connect to Groq.

Step 12 — Issue: Wrong Groq API URL

■ ISSUE 5: 404 Not Found — Wrong URL path

```
■ ERROR: 404: Unknown request URL: POST /openai/chat/completions Please check the URL for typos, or see the docs at https://console.groq.com/docs/
```

The base URL was set to `https://api.groq.com/openai` but Spring AI appends `/chat/completions` making the full URL:

`/openai/chat/completions` — which is wrong for Groq.

Groq's correct path is: `/openai/v1/chat/completions`

■ SOLUTION: Add /v1 to the base URL

```
# Fix the base URL sed -i 's|https://api.groq.com/openai|https://api.groq.com/openai/v1|g' \
~/sameer-ai-platform/services/ai-gateway/src/main/resources/application.properties # Verify
cat ~/sameer-ai-platform/services/ai-gateway/src/main/resources/application.properties
```

Correct URL should be:

```
spring.ai.openai.base-url=https://api.groq.com/openai/v1
```

■ URL fixed — Spring AI now correctly calls `https://api.groq.com/openai/v1/chat/completions`

Step 13 — Issue: Groq Model Deprecated

■ ISSUE 6: Model decommissioned

```
■ ERROR: 400: The model 'llama3-8b-8192' has been decommissioned and is no longer supported.
Please refer to https://console.groq.com/docs/deprecations for a recommendation.
```

The model `llama3-8b-8192` was used initially but Groq has deprecated it. Need to use the newer model name.

■ SOLUTION: Update to current model name

```
# Update model name in application.properties sed -i
's|llama3-8b-8192|llama-3.3-70b-versatile|g' \
~/sameer-ai-platform/services/ai-gateway/src/main/resources/application.properties # Verify
cat ~/sameer-ai-platform/services/ai-gateway/src/main/resources/application.properties
```

Then rebuild and restart:

```
cd ~/sameer-ai-platform/services/ai-gateway mvn clean package -DskipTests -q cd
~/sameer-ai-platform docker compose down docker compose up -d --build sleep 15 curl
http://localhost:8080/api/ai/chat?message=who+are+you
```

■ I am Nuh, an AI assistant created by Sameer Shaik from N.S. — Working!

Step 14 — Final Test & Verification

Test all services:

```
# Check all containers running docker compose ps # Test AI response curl
http://localhost:8080/api/ai/chat?message=who+are+you # Test Python service curl
http://localhost:8000/health # Check WhatsApp bot connected docker compose logs whatsapp-bot
--tail=5
```

■ All 4 containers running: ai-gateway, whatsapp-bot, python-service, open-webui

■ AI Response: I am Nuh, an AI assistant created by Sameer Shaik from N.S.

■ WhatsApp Bot: SameerBot connected — 919441256250:3@s.whatsapp.net

■ Open WebUI accessible at: <http://3.104.66.117:3000>

Final architecture on AWS:

Container	Port	Technology	Status
ai-gateway	8080	Spring Boot + Groq API	Running
whatsapp-bot	—	Node.js + Baileys	Connected
python-service	8000	FastAPI (mute/archive)	Running
open-webui	3000	Open WebUI dashboard	Running

Step 15 — Daily Management Commands

Start everything:

```
cd ~/sameer-ai-platform docker compose up -d
```

Stop everything (save money):

```
docker compose down
```

Check status:

```
docker compose ps
```

View bot logs:

```
docker compose logs -f whatsapp-bot
```

View all logs:

```
docker compose logs -f
```

Restart single container:

```
docker compose restart ai-gateway
```

SSH into EC2:

```
ssh -i ~/Downloads/sameer-nuh-key.pem ubuntu@3.104.66.117
```

Stop EC2 instance (from AWS Console):

EC2 Console → Instances → Select instance → Instance State → Stop

■ Warning: STOP not TERMINATE — Stop saves data, Terminate deletes everything permanently!

Cost reminder:

EC2 State	Compute Cost	Storage Cost
-----------	--------------	--------------

Running	~\$0.12/hr (~\$86/mo)	~\$2.40/mo (30GB)
Stopped	FREE	~\$2.40/mo (30GB)
Terminated	FREE	FREE (data deleted!)

Issues Summary — Quick Reference

#	Issue	Root Cause	Fix
1	docker-compose-plugin not found	Ubuntu 26.04 default repo lacks it	Add Docker official repository manually
2	SCP broken pipe transferring project	Thousands of small WhatsApp audio files	Zip files first, then transfer single zip file
3	Spring cannot reach Ollama (500 error)	Docker.internal does not work inside EC2	Use EC2 instance gateway IP 172.18.0.1 instead
4	AI response takes 60-120 seconds	No GPU on EC2 — CPU-only Ollama	Switch to Groq API (free, 1-2 second response)
5	404 Wrong Groq URL	base-url missing /v1 path segment	Set base-url to https://api.groq.com/openai/v1
6	Groq model decommissioned	llama3-8b-8192 deprecated by Groq	Use llama-3.3-70b-versatile instead